



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Mohd. Razia Alangir Banu
Student Name	Y. Yashwanth
Roll Number	2520030506
Branch	CSE
Course Outcome	

Case Study Topic:

Work Done Summary:

In this case study, we designed and implemented a **Wavelet Tree** data structure optimized for 23andMe's genomic-sequence pipeline to evaluate high-throughput range-rank queries over massive DNA sequence sets.

Scale Challenge: The analytics pipeline manages 100 million (10^8) sample DNA strings, with each sample encompassing approximately 3×10^9 bases over the 4-letter alphabet $\Sigma = \{A, C, G, T\}$. A naive character scan across this data scale is computationally impossible.

Algorithmic Solution: To solve this, a Wavelet Tree was structured over the alphabet $\Sigma = \{A, C, G, T\}$ to process rank-in-range operations in optimal $O(\log \sigma)$ time, where the alphabet size $\sigma = 4$. This architecture collapses the problem space into exactly 2 levels of compressed bit-vectors.

Miniature Evaluation: The implementation was validated using a sample sequence $S = \text{ACGTACGTACGTACGT}$ (length 16) to execute the analytical query: *"Count the number of times the character 'A' appears in the substring range $S[3..10]$."*

Implementation Details:

· **Alphabet Partitioning:** Structured the tree to systematically split the alphabet in half at each tier: the top level partitions $\Sigma = \{A, C\}$ vs $\Sigma = \{G, T\}$, while the bottom level separates A vs C and G vs T .

Root-Level Encoding: Formed the master root level bit-vector of length 16 where characters $\Sigma = \{A, C\}$ are mapped to bit 0 and characters $\Sigma = \{G, T\}$ are mapped to bit 1.

Child Vector Generation: * Generated the **Left Child** bit-vector, filtering only the characters where the root-bit was 0\$ (\$\{A, C\}\$), assigning bit 0 = A\$ and bit 1 = C\$. Generated the **Right Child** bit-vector, filtering only the characters where the root-bit was 1\$ (\$\{G, T\}\$), assigning bit 0 = G\$ and bit 1 = T\$.

Range-Rank Algorithm: Engineered an index-tracking algorithm that descends the tree to resolve substring bounds. To count 'A's in $S[i..j]$, the query descends left from the root using $\text{bit}=0$. The surviving sub-range updates index boundaries to point into the left child, where counting $\text{bit}=0$ completely isolates the exact occurrences of A\$.

Result: Screenshots / Output

```
Run: WaveletTreePipeline x
Run: WaveletTreePipeline.main()
"C:\Program Files\Java\jdk-17\bin\java.exe" --preview-feature WaveletTreePipeline

Act: WAVELET TREE INITIALIZATION (Alphabet: {A, C, G, T})
Sequence S: A C G T A C G T A C T A C G T
Indices: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Bit-Vectors That Occurred:
-----
Root Level (0 = A or C, 1 = G or T)
Balances: [0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
Indices: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Left child (chars where root-bit=0: A or C)      Right child (chars where root-bit=1: G or T)
Balances: [0, 1, 0, 1, 0, 1, 0, 1]              Balances: [0, 1, 0, 1, 0, 1, 0, 1]
Indices:  A/0 C/1 A/2 C/3 A/4 C/5 A/6 C/7          Indices:  G/0 T/1 G/2 T/3 G/4 T/5 G/6 T/7

FINAL WAVELET TREE (Hierarchical Deconstruction)
          [ Root ]
          /      \
    0 = {A,C} | 1 = {G,T}
    /      \
[ Left Child ] [ Right Child ]
0 = 'A' | 1 = 'C'  0 = 'G' | 1 = 'T'

-----
EXECUTING RANGE-RANK QUERY:
Target Substring Range: S[3..10] ("TACGTACG")
Analytical Query: Count 'A's inside sequence segment

Execution Steps:
1. Evaluate range [3..10] at Root level bit-vector.
2. Map indices to Left Child by counting '0' bits up to boundaries.
   - New Left Child sub-range computed: [1..5]
3. Scan Left Child bit-vector within [1..5] matching target bit=0 ('A').
   - Matches discovered at Left Child indices: 2, 4

QUERY COMPLETE
Resulting 'A' occurrences found in range: 2
Time Complexity (Worst Case):  $O(\log \sigma) \rightarrow O(2)$  operations for Alphabet Size 4

Process finished with exit code 0
=====
```

Conclusion & Analysis

Query Verification: For the sample range query $S[3..10]$ which isolates the substring sequence `TACGTACG`, the Wavelet Tree successfully evaluated and returned a total count of 2 'A's.

Time Complexity: * **Wavelet Tree Insertions/Construction:** $O(n \log \sigma)$

Range-Rank Query: $O(\log \sigma)$ where $\sigma = 4$

Comparative Efficiency: Compared to a standard sequential scan ($O(n)$) which completely fails under large-scale workloads (10^8 samples $\times 3 \times 10^9$ bases), the Wavelet Tree performs significantly better by restricting indexing lookups to simple bitwise rank operations over exactly $\log \sigma$ layers.

GitHub Link: <https://github.com/Yashwanthwhy/DSA-2/blob/main/CaseStudy-2.docx>

Student Signature	Faculty Signature
-------------------	-------------------

.